AES Data Scientist Take-Home Assessment

Proof-of-concept technical exercise for applied data science, ML judgment, and communication

Time expectation and intent

Please spend no more than **5 hours** on this assignment. You will have **3 business days** to complete it. We are evaluating how you structure ambiguity, identify the highest-leverage 20%, make defensible trade-offs, understand the solution you build, and explain your work clearly. A small, well-reasoned proof of concept is better than a broad but shallow solution.

Overview

AES operates renewable and conventional power assets across diverse markets. Operators, asset managers, commercial teams, and engineering teams need to make timely decisions from incomplete, noisy, and often disconnected data. This assignment asks you to design and prototype a practical decision-support workflow for one realistic AES problem: identifying which renewable assets most need attention and explaining why.

You are not expected to solve the full production problem. Your goal is to create a proof of concept that demonstrates the core analytical logic, makes pragmatic assumptions, and shows how the solution could evolve into a production-quality tool.

Scenario: Renewable Asset Performance Triage

AES has a portfolio of solar, wind, and battery assets. Each asset produces operational telemetry and receives weather, availability, curtailment, market, and maintenance signals. The operations team wants a daily triage view that answers:

- Which assets appear to be underperforming relative to expectation?
- Which potential causes are most plausible: weather, curtailment, outage, degradation, sensor/data issue, or market/dispatch constraint?
- Which issues should be prioritized first based on estimated financial or operational impact?
- What should a human operator or asset manager inspect next?

Right now, different teams answer these questions using separate dashboards, spreadsheets, and ad hoc analysis. Leadership wants to understand whether a data science workflow could turn these signals into a practical daily decision-support product.

Your Task

Create a proof-of-concept solution for the Renewable Asset Performance Triage problem. You may use Python, notebooks, SQL, BI mockups, diagrams, pseudocode, or a written design. Working code is encouraged but not required. If you write code, it may use synthetic data that you generate yourself.

Assume you have access to the following data sources. You do not need to model every field; choose the subset that matters most for a credible POC.

Data source	Example fields	Typical grain
Asset metadata	asset_id, technology, region, capacity_mw, COD, owner, market, OEM	One row per asset
Operational telemetry	timestamp, asset_id, generation_mwh, availability_pct, inverter/turbine status, battery SOC	5-min to hourly
Weather and resource	irradiance, wind_speed, temperature, humidity, forecast_resource, actual_resource	Hourly
Market/dispatch	price, dispatch_instruction, curtailment_flag, congestion indicator, interconnection limit	Hourly
Maintenance logs	work_order_id, asset_id, opened_at, closed_at, category, free-text notes, severity	Event-level
Financial assumptions	expected_price, PPA rate, merchant exposure, penalty/bonus terms, estimated lost revenue	Hourly or monthly

Use of AI tools

You may use AI tools, open-source packages, and public documentation. Be prepared to explain every major design, modeling, and implementation choice. Please include a brief note describing where you used AI assistance and how you validated the output.

Part 1: Problem Decomposition and POC Scope

Suggested time: 45-60 minutes

Frame the problem in a way that would let a small team build a useful first version quickly.

1. **Problem statement:** Define the specific decision your POC supports. What question will it answer, for whom, and how often?
2. **Scope boundaries:** What are you intentionally not solving in the first version? Why?
3. **Success criteria:** What would make the POC valuable enough to continue investing in? Include both analytical and business/user success measures.
4. **Key assumptions:** List the assumptions you are making about data availability, quality, latency, users, and operational constraints.

Part 2: Analytical / ML Approach

Suggested time: 90-120 minutes

Design the analytical core of the POC. The solution may be simple. We care more about judgment and fit-for-purpose reasoning than use of advanced methods.

Your response should address:

- **Expected performance baseline:** How would you estimate what generation, availability, revenue, or dispatch should have been under normal conditions?
- **Anomaly / underperformance detection:** How would you identify meaningful deviations rather than noise?
- **Cause attribution:** How would you distinguish likely weather/resource issues from curtailment, outages, degradation, data problems, or market constraints?
- **Prioritization:** How would you rank issues by operational or financial importance?
- **Human-in-the-loop design:** How should operators validate, override, or provide feedback to improve the system over time?
- **Trade-offs:** Compare at least two possible approaches. For example: rules vs. statistical baselines, gradient-boosted models vs. physics-informed methods, batch workflow vs. near-real-time alerts, interpretable models vs. higher-performing black-box models.

Optional coding component:

Build a small POC using synthetic data or a simplified public/simulated dataset. A good POC might generate hourly asset data for 10-50 assets, create an expected-output baseline, flag anomalies, rank the top issues, and produce a simple table or chart explaining the result. The code does not need to be production-ready.

Part 3: Product and System Design

Suggested time: 60-75 minutes

Describe how the POC could become a practical internal data product. You do not need to design every production detail, but you should show that you understand the path from notebook to reliable decision support.

- **Architecture:** Show or describe the end-to-end workflow from raw data to daily triage output. Include ingestion, feature creation, model or rule execution, storage, visualization, and alerting.
- **Technology choices:** Recommend a plausible AES-friendly stack. You may reference tools such as BigQuery, Vertex AI, Databricks, Airflow, GitHub, Looker/Power BI, MLflow, dbt, Python, or other tools you believe fit. Explain why.
- **Data quality:** Identify the most important data quality risks and how you would detect them.
- **Monitoring:** What should be monitored once this is live? Include model performance, data drift, alert volume, false positives, and user adoption.
- **Security and governance:** What data access, auditability, or model governance considerations matter for this use case?

Part 4: Communication and Leadership

Suggested time: 45-60 minutes

This role requires turning ambiguous analytical work into clear decisions. Include the following two communication artifacts:

1. **Executive summary:** Write a concise half-page summary for a non-technical VP of Operations or Asset Management. Focus on the business value, what the first version will and will not do, the main risks, and what you need from stakeholders.
2. **Team implementation plan:** Imagine you are working with one data engineer, one data scientist, and one business SME. How would you split the work over the first 4-6 weeks? What practices or guardrails would you put in place so the team moves quickly without creating an unmaintainable science project?

Part 5: Open-Ended Extension

Suggested time: 20-30 minutes

Name one additional high-impact data science or GenAI opportunity at AES that you would want to explore after completing this POC. Explain why it could matter, what data you would need, and what would make you decide not to pursue it.

Submission Format

Please submit a single PDF or Google Doc. If you include code, provide a GitHub repository, notebook, or zipped folder. Your submission should be readable without running the code.

Recommended structure:

- 1-2 page written summary of your approach and key trade-offs
- 1 architecture diagram or workflow sketch
- 1 table or chart showing the POC output, such as a ranked issue list
- Optional notebook or script demonstrating the analytical logic
- Brief note on AI/tool usage and how you validated the work

What good looks like

Do not spend time creating a polished enterprise application. A simple notebook plus a clear memo is completely acceptable. We value clear reasoning, practical prioritization, and honest treatment of uncertainty.

Evaluation Criteria

Dimension	What we are looking for
Problem judgment	Does the candidate deconstruct the ambiguous problem, choose a sensible first slice, and avoid overbuilding?
Technical rigor	Is the analytical approach sound, explainable, and grounded in realistic data limitations?
Trade-off reasoning	Do they compare options and make deliberate choices around complexity, latency, accuracy, cost, and maintainability?
POC execution	If they build something, does it demonstrate the core logic cleanly? If they do not code, is the design concrete enough to build?
Communication clarity	Can they explain the solution to both technical teammates and operational stakeholders?
Ownership and leadership	Do they show awareness of deployment, monitoring, adoption, feedback loops, and team practices?
AES context	Do they engage with energy-sector realities such as asset operations, intermittency, curtailment, market impacts, safety, and governance?

Optional Interview Follow-Up Guide

Use the live follow-up discussion to test whether the candidate actually understands the work they submitted, especially if they used AI or heavy external assistance.

Suggested follow-up questions:

- What did you decide not to include in the POC, and what made that the right cut?
- Walk us through one asset that your system flagged. Why was it flagged, and what would you ask an operator to check next?
- Where would your approach break down if scaled from 10 assets to 1,000 assets?
- How would you know whether the system is creating real value rather than just more alerts?
- What is the simplest version you would ship in 4 weeks? What would you defer for 6 months?
- Which part of your solution would you trust least in production, and how would you monitor it?
- What did AI help you with, and how did you verify that the output was correct?

Interviewer scoring rubric

Score	Interpretation
5 - Excellent	Clearly identifies the highest-leverage slice, uses an appropriate simple approach, explains trade-offs, anticipates deployment/adoption risks, and communicates crisply.
4 - Strong	Solid analytical design with reasonable scope and good communication; minor gaps in operationalization or trade-off depth.
3 - Mixed	Understands the basic problem but either overbuilds, under-specifies implementation, or misses important energy/operations constraints.
2 - Weak	Generic data science answer, little prioritization, unclear business value, or poor understanding of the proposed method.
1 - Not acceptable	Cannot explain their own work, relies on buzzwords, or proposes an approach that would be misleading or unsafe in an operational setting.